Recommended Practices

1 Naming Conventions

1.1 File Naming

- A file must contain only one design unit, contained in a single module (from `module` to `endmodule`).
- All Verilog files must have the extension `.v`.

1.2 Variables, Signals, and Other Objects

- Names must use only alphanumeric characters and underscores.
- Names must start with a letter, not a number or underscore.
- Names must be unique regardless of case.
- Parameter and constant names must be uppercase (for example, `PI` and `DELAY`).
- Signal and variable names must be lowercase and meaningful.
- A constant name must describe its purpose (for example, `rega_enable`).
- Construct names, such as module, function, and task names, must also be meaningful.
- Use underscores in names made up of multiple words (for example, `load_input`).
- Use the suffix `_b` for active-low signals (for example, `clear_b`).
- If a clocking signal's name does not already contain `clock` or `clk`, use the suffix `_clk` (for example, `bus_clk`).
- Do not bundle unrelated signals into buses.

2 Comments

Comments are required to describe the functionality of a design unit. Each file must have a consistently formatted header that:

- Names the file and the highest-level construct it contains.

- Identifies the originating section or department, the author, and the author's email address.
- Records release history whenever a change is registered.
- States the module's purpose and explains its functionality.
- Describes the parameters used in the construct.
- Documents the number of clock domains, the clocking strategy, and critical timing, including external timing relationships.

3 Coding Style

Consistent style makes code easier to understand and maintain.

- Indent code consistently in a tabular format. Use a constant indentation of 2 to 4 spaces; do not use tabs, whose tab stops can vary between systems.
- Use spaces and blank lines to improve readability.
- Put no more than one Verilog statement on a line.
- Use single-line comments beginning with `//`; avoid multi-line comments using `/* ... */`.
- Keep lines shorter than 80 characters to avoid wrapping.
- Declare one port per line and follow each port listing with a descriptive comment.

4 Module Partitioning

Partitioning reduces complexity and helps minimize the chances of error.

- Procedures, tasks, and functions must not access or modify signals or variables that are not passed as parameters.
- If using a gated clock, an internally generated clock, or both edges of a clock, keep the clock-generation circuitry in a separate module at the top level or at the same logical hierarchy level as the block whose clocks it controls.
- Partition separate clock domains (for example, slow and fast clocks) into separate blocks.
- Partition the design to minimize the number of interface signals.
- Do not mix structural and behavioral RTL code within one construct.
- Partition state machines and asynchronous logic into separate blocks.

5 General Coding Techniques

- A condition expression must have a one-bit value. For example, replace:

```
if (bus) data_avail = 1;
```

  with an explicit comparison:

```
if (bus > 0) data_avail = 1;
```

- Do not assign signals to `x`.
- Do not infer latches in functions; a function is expected to synthesize combinational logic.
- Operand sizes should match.
- Use parentheses in complex expressions.

6 Guidelines for Synthesis

The following practices help keep RTL synthesizable and its behavior explicit.

- Every `always` block for combinational logic or a latch must have a sensitivity list containing all of its input signals.
- Use only one clock in an `always` block.
- Do not use `wait` or `#delay` statements. A `wait(expression)` evaluates its expression and suspends execution while the expression is false.
- Specify conditional expressions completely: assign a value to a variable under every condition to avoid unintended latches.
- Do not use the `initial` statement.
- Do not use expressions in port connections.
- Do not use Verilog user-defined primitives.
- Use non-blocking assignments in edge-sensitive constructs.
- Declare internal `wire`s after the port I/O declarations at the top of the module.
- Use explicit port references in module instantiations. For example:

```
full_adder FA (
    .sum(S),
    .cout(Co),
    .in1(A),
    .in2(B),
    .cin(Ci)
);
```

- Do not use `casex`; synthesis treats `x` and `z` states as don't-cares.
- Use parameters for state encoding.
- Use only `input` or `output` ports; avoid bidirectional `inout` ports.
- In procedural blocks, use blocking assignments to model combinational logic.
- Include a `default` case in every combinational-logic `case` statement description.